

LOW-LEVEL DESIGN

AI in Air-Gapped Environments — Lessons Learned & Best Practices (LLD)

Implementation patterns, algorithms, configuration surfaces, and test strategy for the practices in the companion HLD, as built in ION

DOCUMENT TYPE

LOW-LEVEL DESIGN

DATE

2026-07-15

OWNER

ION Maintainer (ixion36)

ION VERSION

v0.51.0

CLASSIFICATION

PUBLIC

AUDIENCE

Engineering teams implementing AI capability in disconnected estates; technical reviewers

Contents

2.1 Deployment shape

2.2 Client discipline

2.3 Sizing lesson

3.1 Layer model

3.2 Budget algorithm

3.3 Output contract enforcement

5.1 Server-gathered evidence blocks

5.2 Mechanical citation validation

6.1 Store

6.2 Staleness & re-embed triggers

6.3 Asymmetric task prefixes (measured)

6.4 Chunking (L5)

6.5 Background producer pattern

6.6 Deterministic-first retrieval (P8)

7.1 Ledger

7.2 Eval harness

1. Introduction

This Low-Level Design is the implementation companion to `docs/AI_AIRGAP_BEST_PRACTICES_HLD.md`. Where the HLD states principles (P1–P13) and lessons (L1–L10), this document shows how ION implements them: concrete algorithms, configuration surfaces, schema patterns, and the tests that pin the behaviour. File references point into the ION codebase as the worked example; the patterns transfer to any stack.

2. Local inference host (P1, P7)

2.1 Deployment shape

- One **Ollama** container co-deployed with the application (compose service on the internal network only; the port is never published beyond it).
- One generation model (security-domain-tuned 8B instruct, Q4 GGUF) and one embedding model (`nomic-embed-text`, 768-dim) served by the same host.
- **Air-gap provisioning:** model weights ship as GGUF files through the offline artefact channel and are side-loaded (`scripts/ollama_import_gguf.sh` builds a Modelfile and registers the model locally). The application only ever references a model *name*, so registry-pulled and side-loaded models are interchangeable.

2.2 Client discipline

Every call site goes through one service wrapper (`ollama_service.py` / `embedding_service.py`) that owns:

- **Explicit context window.** The request sets `num_ctx` from configuration (`ION_OLLAMA_NUM_CTX`, default 16384). *Never* rely on the runtime default — Ollama's silent ~4096 default front-truncated ION's prompts for multiple releases (L2), destroying the output contract that lives at the end of the prompt.
- **Timeouts sized to the operation** (chat vs embedding) and **circuit breakers** fed by transport-level success/failure so a dead host fast-fails instead of queueing.
- **Return-None-never-raise** for embeddings: `embed()` returns `None` on any failure; every caller treats `None` as "skip and retry next tick". This one convention is what makes default-on RAG safe (P7).
- **Response-shape tolerance:** the wrapper accepts both current and legacy API response shapes explicitly. Reading a single hardcoded key is how the eval harness silently died (L3).

2.3 Sizing lesson

8B Q4 inference is impractical on CPU-only hosts (minutes per response); it is comfortable on a single modest discrete GPU. Embeddings (`omic-embed-text`) are fine on CPU. Practical estate guidance: GPU for the generation host; embedding-only deployments can run anywhere. Design features so the *embedding* tier (similarity, retrieval) still delivers value on estates without generation-capable hardware.

3. Prompt assembly: layers under a token budget (P3, P9)

3.1 Layer model

ION's system prompt is assembled from fixed and optional layers

(`alert_prompt_service.render_system_prompt`):

```
FIXED 1. base persona (security-analyst system prompt)
FIXED 2. per-alert-type investigation guide (deterministic 5-tier template match:
      rule_id → regex → ATT&CK technique → tactic → rule group)
OPTIONAL 3. KB RAG chunks (priority 1 – dropped last)
OPTIONAL 4. gold exemplars (priority 2)
OPTIONAL 5. matched playbooks (priority 3)
OPTIONAL 6. keyword-matched skills (priority 4 – dropped first)
FIXED 7. output contract (ALWAYS appended, NEVER budget-gated)
```

3.2 Budget algorithm

```
budget    = ION_SYSTEM_PROMPT_TOKEN_BUDGET (default 3800; floor 1500)
estimate  = len(text) / 4          # chars-per-token, conservative for mixed prose+IOC
fixed     = est(base) + est(guide) + est(contract)
remaining = budget - fixed
for layer in optional_layers_in_priority_order:
    block = fetch(layer)           # any failure → empty block
    if est(block) ≤ remaining:
        append(block); remaining -= est(block)
    else:
        log.debug("layer dropped: needs X, only Y remaining") # loud, not silent
append(contract)                  # unconditional, last
```

Rules that matter (each traces to an incident):

- The **contract is exempt** from budgeting and always terminates the prompt (L1).
- Drops are **logged** — a layer silently vanishing is how quality sags unnoticed (P9).
- Layer fetchers are individually try/except-wrapped: a retrieval failure costs its layer, never the prompt.

3.3 Output contract enforcement

- The contract demands a JSON envelope; the verdict field must be a member of a server-side enum (ION: `CaseClosureReason`) shared with the human closure workflow, so AI suggestions and human decisions are 1:1 comparable in the ledger (§7).
- The parser validates after every call: malformed JSON → fallback extraction → else "inconclusive". Parse failure is **never** surfaced as analysis text.
- Confidence arrives as both a tier (low/medium/high) and an integer (0–100); the integer drives thresholds (§8).
- Length caps at the persistence layer: stored prompt snapshots and raw responses are hard-capped (100k chars) so a runaway generation cannot balloon the database (P10).

4. Input handling (P4)

- All telemetry-derived content is wrapped in a delimited `<input_data>` block with an instruction that its contents are data to analyse, not instructions to follow.
- A value sanitiser runs over interpolated fields; rendered output is HTML-escaped at the UI layer under a strict CSP (defence-in-depth: even a successful injection that produces malicious markup does not execute).
- **Nothing adversary-influenced is used as a prompt *instruction*** — rule names, alert reasons, and file contents appear only inside the data block.
- RAG corpus discipline: only first-party operator content (KB articles, playbooks, human-closed cases) is embedded for retrieval. Raw alert text is embedded solely as *query* vectors (transient, never stored as corpus), so retrieval cannot launder adversary text into future prompts.

5. Evidence grounding & citation validation (P5)

5.1 Server-gathered evidence blocks

Pattern (used by the closure-note rewriter, `_gather_closure_evidence`): the server assembles a bounded plain-text evidence block from data the caller is already authorised to read — entity fields, extracted observables, analyst notes, the latest *decisive* AI investigation summary, TI-enrichment digests. Bounds are structural: per-section clips (300–800 chars) plus a whole-block cap (3000 chars), so no single verbose field evicts the rest (L5 generalised). The prompt frames this block as "the ONLY facts you may cite".

Two grounding rules learned the hard way:

- **Decisive-verdict filter:** inconclusive AI summaries are boilerplate ("insufficient evidence to determine...") — near-identical across unrelated items. Embedding or citing them pollutes both retrieval and generation (L9). Filter to decisive verdicts wherever AI output feeds back into context.

- **Precedent demotion:** similar-item precedent may contribute at most one trailing sentence and may never open the response — otherwise, with a thin draft, precedent becomes the entire answer (L4).

5.2 Mechanical citation validation

Pattern (auto-investigation): the server builds a numbered evidence ledger, the contract requires every finding to cite ledger IDs, and post-processing enforces:

```
for finding in response.findings:
    finding.citations = [c for c in finding.citations if c in ledger_ids]
    if not finding.citations: drop(finding)
if response.verdict is decisive and no findings survived:
    response.verdict = "inconclusive"          # downgrade, never trust unsupported
if response.playbook_id not in offered_candidates: discard(playbook_id)
```

The model cannot cite what the server didn't provide, and cannot keep a conclusion its surviving citations don't support.

6. Retrieval implementation (P8, P9)

6.1 Store

- **pgvector in the primary Postgres** — one extension (`CREATE EXTENSION vector` , run *before* `create_all` so vector columns can be created), HNSW indexes via raw DDL helpers called from init (SQLAlchemy's Index() cannot emit `USING hnsw`).
- One embedding table per corpus, all following the same shape: `entity_id (FK, CASCADE)` , `embedding VECTOR(768)` , `model_name` , `embedded_at` , `source_text_hash` . Guarded pgvector import (String fallback) keeps modules importable on non-pgvector dev environments.

6.2 Staleness & re-embed triggers

A row is fresh iff `source_text_hash == sha256(current_source_text)` and `model_name == current_model_tag` . The `model_tag` encodes the embedding model *plus* the prompt-prefix scheme (`nomic-embed-text+tp1`) — flipping any regime marker makes the whole corpus stale and re-embeds it once, under one consistent scheme. Chunking parameters fold a scheme marker (`c1`) into the hash input for the same reason. **Embeddings are regenerable derived data:** migrations may simply drop a vector table and let the loop rebuild it — ION's chunk-level migration did exactly this.

6.3 Asymmetric task prefixes (measured)

`nomic-embed-text` is an asymmetric retrieval model: corpus text embeds under `search_document:` , live queries under `search_query:` . ION measured this on its real KB (80 tagged docs, leave-one-out): **+57%**

nDCG@10, +58% MAP over prefix-less embedding. Two implementation notes: gate it (a model that doesn't understand prefixes degrades), and validate the mode argument even when disabled so a typo'd call site fails loudly in tests rather than silently mis-embedding.

6.4 Chunking (L5)

Whole-document vectors fail twice: the embedding window truncates long docs' tails, and retrieval can only return the *document*, so the prompt quotes the doc head rather than the matched passage. ION's chunker: greedy paragraph packing to ~1600 chars (~400 tokens, safely inside the window), overlap carry-over only on hard splits of oversized paragraphs, 64-chunk cap per doc, chunk text stored alongside its vector. Retrieval ranks chunks, dedups to the best chunk per document (over-fetch $k \times 4$, cut at the similarity threshold — rows are distance-ordered so the first miss ends the scan), and injects the **matched chunk's own text**.

Write discipline: a document's chunk set is replaced atomically — embed all chunks first, then delete-and-insert in one transaction; any mid-document embed failure skips the whole document until the next tick. Batch budgets count *chunk* embeds with an at-least-one-document guarantee so one long article can't starve the queue.

6.5 Background producer pattern

Every corpus has one producer loop with the same skeleton: Postgres advisory lock for cross-worker leader election (distinct lock ID per loop — ION keeps a registry of `LOCK_*` constants), interval ticks, per-tick batch limit, skip-fresh check, back-off after 3 consecutive embed failures, `last_run` introspection for health endpoints. Lifecycle rule: an entity flipped inactive/disabled has its vectors **evicted** on the next tick — retrieval must never surface a disabled procedure (L10).

6.6 Deterministic-first retrieval (P8)

The playbook layer's selection order:

```
hits = repository.find_matching(rule_name, severity, mitre) # structured match
if hits: return top_by_priority(hits)                       # no model involved
else:    return cosine_top_k(query_vec, threshold=0.65, k=2) # fallback only
```

The structured arm works with no Ollama at all — on a degraded estate the feature keeps functioning at reduced recall. Similarity thresholds are per-corpus (ION: 0.65 for topic-level KB, 0.70 for case exemplars) — looser where semantic overlap is broader by design.

7. Feedback ledger & offline evaluation (P6)

7.1 Ledger

- Dual-write: once when the AI suggestion fires, once when the human closes — pairing `(ai_verdict, human_verdict, agreement)` per alert/template.
- **Read contract:** the ledger is append-style; readers MUST dedup with `MAX(id)` per `(alert_id, template_id)`. Pin the contract with a test — ION's detection-health dashboard depends on it.
- Derived views: per-rule agreement rates, disagreement drill-downs, threshold-tuning inputs.

7.2 Eval harness

- Replays **stored prompt snapshots** (not re-assembled prompts — you are evaluating the model/config change, not incidental pipeline drift) against a candidate configuration, forcing JSON output to match the live path, and scores P/R/F1 against the human ledger verdicts.
- Runs under its own advisory lock as a background job; results persist per-run for comparison.
- **Test the harness itself** (L3): assert non-degenerate outputs on a known corpus; alarm when every sample abstains. Run evals on hardware representative of production — CPU-only dev boxes cannot exercise an 8B model meaningfully.

8. Confidence thresholds & escalation (P12)

- Per-template `confidence_threshold_override` with a global default; below threshold, the suggestion is suppressed and a visible "auto-escalated" state is set (`bob_escalation_badge`) — the analyst sees *that the AI stood down*, not a low-quality guess.
- A circuit breaker on sustained low confidence stops suggestion generation entirely until quality recovers.
- Wire-through test matters: ION shipped the override column long before anything read it (dead lever). Pin the path from configuration to behaviour with a test the day the column lands.

9. AI data-at-rest inventory (P10)

STORE	CONTENT	CONTROL
Prompt snapshots	Full assembled prompt per investigation	100k-char hard cap; single-tenant DB
Raw responses	Unparsed model output	100k-char hard cap

STORE	CONTENT	CONTROL
Reasoning text	Model's analyst-facing explanation	Storage gate (<code>ION_BOB_STORE_REASONING</code>); opt-out also suppresses previously stored rows at the response layer — no backfill purge needed
Embeddings (cases/KB/playbooks)	768-dim vectors + source hashes (+ chunk text for KB)	Regenerable — droppable in migrations; CASCADE on entity delete
Feedback ledger	AI vs human verdicts	Append-style + dedup read contract
Eval runs	Replay scores per configuration	Operator-only surface

Every row of this table appeared in a release's security-assessment delta when it was introduced.

10. Operations

- **Packaging:** application image + model GGUFs travel the same offline channel; the GGUF import script registers models by name so application config is estate-independent. Pin model files by checksum in the transfer manifest (P11).
- **First boot / corpus build:** embedding loops build the corpus incrementally (batched per tick) — expect retrieval layers to warm up over the first hour on a large estate; deterministic layers work immediately.
- **Health:** each loop exposes last-run/last-result introspection; the integrations page shows model-host reachability; scrape-time gauges are exported only via the opt-in first-party metrics endpoint (P13).
- **Model swap runbook:** import new GGUF → run offline eval against replayed prompts → compare P/R/F1 per template → flip the model name in config → embedding `model_tag` mismatch triggers automatic corpus re-embed → monitor agreement rates in the ledger for regression.

11. Test strategy

Patterns that caught real bugs, in descending value:

1. **Prompt-structure pinning:** assert the assembled prompt contains/omits the exact sections, in order, under each gate combination — with a stub model client capturing `last_prompt`. Cheap, catches contract drift, runs with no model.
2. **Contract tests on read paths:** the ledger dedup rule, the enum-pinned verdict validation, the response-shape tolerance — each pinned by a test named for the contract.
3. **Loop lifecycle tests:** fresh-skip, hash-change re-embed, model-tag re-embed, atomic replace (no partial chunk sets), inactive eviction — run against SQLite with a stub embedder (vector types serialise fine; skip only the cosine-query tests that need pgvector).
4. **Degradation tests:** every AI feature with the model host absent → clean no-op, correct HTTP codes, no queue growth.

5. **Gate-default agreement tests:** the per-call gate and the loop-start gate for a feature must share one default — pin them equal, or the loop spins while calls no-op (caught live in ION).
6. **Real-corpus retrieval benchmarks** for embedding changes (leave-one-out nDCG/MAP) — synthetic probes were inconclusive where the real corpus showed +57%.

12. Configuration reference (as built in ION)

VARIABLE	DEFAULT	PURPOSE
<code>ION_OLLAMA_URL</code> / <code>ION_OLLAMA_MODEL</code> / <code>ION_OLLAMA_TIMEOUT</code>	local host / security-tuned 8B / per-op	Inference host binding
<code>ION_OLLAMA_NUM_CTX</code>	16384	Explicit context window — never trust runtime default (L2)
<code>ION_SYSTEM_PROMPT_TOKEN_BUDGET</code>	3800	Prompt-assembly budget (§3.2)
<code>ION_EMBEDDING_ENABLED</code>	on	Master embedding gate (graceful no-op when host absent)
<code>ION_EMBEDDING_MODEL</code>	nomic-embed-text	Embedding model name
<code>ION_EMBEDDING_TASK_PREFIX</code>	on	Asymmetric task prefixes (§6.3); encoded in <code>model_tag</code>
<code>ION_KB_RAG_ENABLED</code> / <code>ION_FEW_SHOT_EXEMPLARS_ENABLED</code> / <code>ION_PLAYBOOK_RAG_ENABLED</code>	on	Per-layer RAG gates
<code>ION_*_EMBEDDING_INTERVAL_S</code> / <code>ION_KB_EMBEDDING_BATCH</code>	300–600s / 40 chunks	Producer-loop pacing
<code>ION_BOB_STORE_REASONING</code>	on	Reasoning-text data-at-rest gate (§9)
<code>ION_MCP_ENABLED</code>	off	AI tool surface — closed by default, permission-checked per tool
<code>ION_METRICS_ENABLED</code> / <code>ION_APM_*</code>	off	First-party observability, opt-in only (P13)

Gate philosophy: retrieval/quality layers default **on** (safe via graceful degradation); new *surfaces* (tool endpoints, telemetry) default **off**.